

# 2D Systolic Arrays for AI Acceleration

Research review: prior work, technical limitations, specific architectural changes, and open research opportunities

Prepared 31 August 2026

**Purpose.** This report reviews conventional two-dimensional (2D) systolic arrays used for matrix multiplication and neural-network acceleration. It focuses on limitations that reduce delivered performance or energy efficiency in real workloads, the concrete changes proposed in published work, and a research direction that remains defensible after considering existing literature.

**Scope note.** “All work” in this area is too broad for a single report. The review instead covers the principal architectural lines: dense fixed-dataflow arrays; dataflow flexibility; array partitioning and scale-out; sparsity; memory-system-aware design; synchronization/data-movement changes; and transformer integration. Numerical results are each specific to the cited paper’s models, baselines, technology assumptions, and workload mix; they should not be read as universal performance guarantees.

## Executive summary

- **Strength.** A 2D systolic array is highly effective for large, dense, regular matrix multiplication. Neighbor-to-neighbor movement enables operand reuse, simple processing elements (PEs), predictable control, and high throughput.
- **Central weakness.** The same fixed spatial schedule creates idle PEs, fill/drain delay, and poor compatibility with irregular shapes, unstructured sparsity, and memory-bound operators. Peak MAC count is therefore often much higher than useful work delivered.
- **State of the field.** Research has already demonstrated flexible dataflow networks, sparse packing and conditional execution, flexible sparse reductions, partitioned “pod” arrays, diagonal dataflow, and system-level memory modeling. A new proposal should not claim these ideas independently as novel.
- **Most promising gap.** A heterogeneous multi-pod array with a compiler/runtime that jointly selects pod shape, dense or structured-sparse mode, dataflow, tiling, and memory layout—evaluated with real DRAM/SRAM effects and end-to-end model latency—targets an unresolved integration problem.

## 1. Background and baseline

For  $C = A \times B$ , where  $A$  has shape  $M \times K$  and  $B$  has shape  $K \times N$ , a conventional 2D array maps output elements  $C_{ij}$  to PEs. In a common output-stationary mapping,  $A$  values move horizontally,  $B$  values move vertically, and each PE accumulates its assigned partial sum. In weight-stationary mapping, weights remain local while activations and/or partial sums stream through. The first Google TPU exemplified the dense matrix-engine approach: its matrix multiplication unit employed a  $256 \times 256$  systolic array, or 65,536 8-bit ALUs [1].

A systolic design optimizes *throughput after the pipeline is full*. It does not automatically minimize one-operation latency, external-memory traffic, or energy for every layer. The dataflow chosen—weight-stationary (WS), output-stationary (OS), input-stationary (IS), or row-stationary (RS)—determines which operand is retained, which links carry traffic, and where reuse occurs [2,3].

## 2. Research landscape

| Line of work                   | Particular change relative to a conventional fixed 2D array                                                  | What it demonstrated / remaining boundary                                                                                                                            |
|--------------------------------|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dense TPU-style array          | Large, regular WS matrix unit with local weight reuse                                                        | Very high dense-GEMM throughput; performance depends on matching layer dimensions to array geometry [1].                                                             |
| Row-stationary CNN accelerator | Maps convolution to preserve several reuse types, not just one stationary tensor                             | Eyeriss showed energy-focused spatial mapping for CNNs; increased mapping/fabric complexity is the trade-off [3].                                                    |
| Configurable dataflow (MAERI)  | Replaces one fixed internal movement/reduction pattern with configurable distribution and reduction networks | Reported 130–283% better utilization than rigid-NoC baselines across evaluated mappings; flexibility has area/energy/control cost [4].                               |
| Sparse-TPU                     | Packs sparse matrices and adds conditional execution to a TPU-like array                                     | Reported 16.08× performance vs. its dense TPU baseline for sparse matrix-vector workloads; nonzero matching and metadata remain costs [5].                           |
| SIGMA                          | Uses a flexible reduction network for sparse irregular GEMM instead of rigid systolic reduction              | Reported 5.7× over systolic-array architectures on irregular sparse matrices; regularity/efficiency is traded for flexibility [6].                                   |
| Scale-out arrays (SOSA)        | Partitions compute into small pods, adds tiling and a pod interconnect                                       | Found 32×32 pods could be 1.5× higher effective throughput/W than 128×128 pods in its single-workload analysis; more pods increase memory/interconnect pressure [7]. |
| DiP                            | Diagonal input movement and permuted weight placement remove conventional WS synchronization FIFOs           | Reported up to 1.49× latency and 1.81× energy improvement on evaluated transformer cases; broad physical/system validation is still needed [8].                      |
| FlexiSAGA                      | Combines dense/sparse dataflows, sparse formats and hardware-aware pruning                                   | Supports seven dense/sparse flows and reports 1.41×–4.28× whole-model sparse-over-dense inference speedups; general transformer coverage remains future work [9].    |
| Memory-aware simulation        | Adds multi-core, sparsity, DRAM timing, data layout, bank conflicts and energy to systolic-array evaluation  | SCALE-Sim v3 showed compute-only rankings can reverse after DRAM stalls are modeled [10].                                                                            |

Table 1. Major directions and the specific modification each makes to the conventional 2D systolic-array model.

## 3. Limitation-by-limitation analysis

### 3.1 Under-utilization from shapes and tiling

A square  $R \times C$  array achieves its theoretical peak only when the tile mapped onto it has useful work for nearly every PE. Real DNN/transformer layers vary in  $M$ ,  $N$ , and  $K$ ; edge tiles, small batches, narrow attention heads, and GEMV-like decode leave rows or columns inactive. Larger arrays also require longer fill/drain phases. SOSA identifies intra-pod dimension mismatch, interconnect limits, and insufficient tiles as three separate utilization losses in multi-pod designs; it reports approximately 22% and 33% utilization for TPU v1 and v4 in its examined workload context [7].

**Specific change to investigate:** Use smaller or partitionable pods, rectangular pods, or heterogeneous pod shapes. Schedule tiles to pods by shape rather than only by availability. The unresolved issue is to improve utilization without turning pod-to-pod routing and memory traffic into the new bottleneck.

### 3.2 One fixed dataflow is rarely optimal

WS minimizes repeated weight fetches; OS keeps partial sums local; IS reuses input activations. But which is best changes with matrix dimensions, batch size, operand reuse, and the capacity/bandwidth of local memory. A fixed fabric is efficient precisely because it is specialized, so it can perform badly outside its favored mapping. Literature on configurable mapping begins from this limitation [2,4].

**Specific change to investigate:** Permit a limited set of selectable dataflows or logical partitions, then choose with a compiler cost model. Avoid full arbitrary flexibility unless its utilization gains exceed its switch, routing, storage, and timing overhead.

### 3.3 Pipeline fill, drain, and boundary synchronization

Interior PEs cannot work until operands travel from array edges. A conventional WS array also needs boundary FIFO synchronization to align streams. For an  $N \times N$  conventional WS design, DiP describes full utilization after  $2N-1$  cycles; its alternative diagonal flow reaches full utilization after  $N$  cycles in its model [8]. This penalty is most visible in small tiles and frequent layer changes.

**Specific change to investigate:** Explore diagonal, skewed, folded, or bypass-capable data movement. Measure not just ideal cycle count but wire length, clock frequency, FIFO area, routing congestion, and the cost of permuting/reordering operands.

### 3.4 Memory bandwidth, locality, and bank conflicts

MAC operations are inexpensive relative to moving data. Systolic reuse lowers traffic but cannot remove the need to stream array boundaries, refill scratchpads, drain outputs, and coordinate many pods. As a PE array grows, compute grows faster than boundary injection capacity; the array becomes increasingly sensitive to memory supply [11]. Data layout and SRAM-bank conflicts can cause stalls even when aggregate bandwidth looks sufficient.

**Specific change to investigate:** Co-design tiling with SRAM bank placement, double-buffering, prefetch, DRAM/HBM page locality, and bandwidth arbitration. SCALE-Sim v3 reports one example where WS was 21% better in compute cycles on six ResNet-18 layers yet OS was 30.1% better once DRAM stalls were included [10].

### 3.5 Unstructured sparsity destroys regular flow

Dense systolic timing assumes matching operands arrive predictably. With arbitrary nonzero patterns, work differs by PE; values need indices; matching/decompression becomes irregular; and PEs stall waiting for partners. Sparse-TPU and SIGMA demonstrate workable alternatives, but their mechanisms explicitly add packing, conditional execution, or flexible reduction/routing [5,6].

**Specific change to investigate:** Prefer block, N:M, tile-aligned, or row/column structured sparsity when retraining permits it. Quantify useful zero-skipping after metadata, load imbalance, routing, and accuracy loss—not only the nominal reduction in MACs.

### 3.6 Full-model and transformer mismatch

The array directly accelerates GEMM, not softmax, layer normalization, activations, embeddings, gathers, reductions, quantization, or KV-cache movement. Large prefill GEMMs map naturally; autoregressive decode is often a small-batch/GEMV and memory-locality problem. A large array can therefore have weak end-to-end benefits if surrounding operators or memory dominate. TiC-SAT explores tight CPU/cache coupling and ISA support for transformer execution rather than treating the array as an isolated engine [12].

**Specific change to investigate:** Use a heterogeneous design: dense-array path for large GEMM, partitionable/smaller pods or vector/MAC-tree path for skinny operations, and SIMD/vector support for reductions and nonlinear functions. Report model-level latency separately for prefill and decode.

### 3.7 Scaling, physical implementation, and reliability

A logical grid hides implementation realities: long global wires, clock distribution, timing closure, NoC contention, thermal density, process variation, and fault handling become harder as dimensions and frequency rise. Aggressive reconfigurable links can harm frequency or consume power. Most architecture papers use modeled area/energy rather than fabricated silicon.

**Specific change to investigate:** Include a physical-aware cost model, use short local links plus hierarchical interconnect, and evaluate area/timing with synthesis or FPGA prototyping. Consider faulty-PE bypass and voltage/thermal-aware scheduling as longer-term extensions.

## 4. Particular changes and maturity

| Problem                         | Change already explored                                                              | Maturity                    | A sharper remaining question                                                                          |
|---------------------------------|--------------------------------------------------------------------------------------|-----------------------------|-------------------------------------------------------------------------------------------------------|
| Mismatched shapes               | Partition into smaller homogeneous pods; scale out with a pod interconnect [7]       | Established concept         | Which <i>heterogeneous rectangular</i> pod mix wins under equal PE count, SRAM, and bandwidth?        |
| Fixed reuse choice              | Configurable distribution/reduction network and flexible mapping [4]                 | Established concept         | Can a restricted, low-overhead 2–3-mode fabric provide most of the gain?                              |
| WS fill/FIFO overhead           | Diagonal data movement and permuted weight placement (DiP) [8]                       | Recent specialized approach | Does the gain survive physical routing, sparse modes, and realistic memory stalls?                    |
| Unstructured zeros              | Packing/conditional execution (Sparse-TPU); flexible sparse reductions (SIGMA) [5,6] | Established but costly      | What structured sparsity granularity maximizes <i>end-to-end</i> speed and accuracy?                  |
| Dense+sparse diversity          | Multiple formats/dataflows plus hardware-aware pruning (FlexiSAGA) [9]               | Recent integrated approach  | Can selection be automated per layer and extended to attention/KV-cache patterns?                     |
| Misleading compute-only results | Cycle-accurate modeling including DRAM, layout, bank conflicts and energy [10]       | Mature tooling direction    | Can the mapper jointly optimize compute mapping and memory placement in real time?                    |
| Non-GEMM bottlenecks            | CPU/cache coupling and specialized transformer integration (TiC-SAT) [12]            | Active system direction     | What is the best split between systolic, vector, and memory-side execution for prefill versus decode? |

Table 2. “Particular changes” should be positioned as extensions or combinations of prior work, not claimed as wholly new ideas.

## 5. Recommended research proposal

A defensible project is an **adaptive heterogeneous multi-pod systolic accelerator with memory-aware mapping and structured-sparse support**. The contribution should be the joint decision process, not simply the existence of rectangular arrays, flexible dataflow, or sparse computation individually.

### Proposed hardware

- Hold total PE count, SRAM capacity, and peak memory bandwidth constant relative to a square-array baseline.
- Use a small heterogeneous pod library—for example, 16×64, 32×32, and 64×16 pods—rather than one large square grid.
- Support dense WS and OS initially; add structured-sparse execution only for tile-aligned or N:M patterns.
- Provide local SRAM banking, double buffers, and a modest hierarchical interconnect. Include a vector/SIMD side unit for softmax, normalization, activation, and quantization.
- Select pod shape, tile dimensions, dataflow, sparse/dense mode, and memory placement with an offline compiler cost model; use a lightweight runtime selector only for dynamic shapes.

### Testable hypotheses

- H1: For a mixed CNN + transformer workload, heterogeneous rectangular pods achieve higher average PE utilization than same-area square pods without increasing total execution energy.
- H2: A memory-aware mapper changes the selected dataflow/tile for a material subset of layers and improves end-to-end latency over a compute-only mapper.
- H3: Moderate structured sparsity can outperform more aggressive unstructured sparsity at equal accuracy because it reduces metadata and load-imbalance overhead.
- H4: A hybrid systolic + vector architecture improves transformer model-level latency versus a systolic-only accelerator, especially for decoding and non-GEMM layers.

### Evaluation plan

- Baselines: equal-area 64×64 and 128×128 WS arrays; 32×32 homogeneous pod array; optional OS baseline.
- Workloads: ResNet-50/MobileNet; a vision transformer; BERT-base; GPT-style layers. Report transformer prefill and decode separately.

- Metrics: PE utilization; total cycles; DRAM/SRAM bandwidth; bank-conflict and memory-stall cycles; pJ/op and total energy; synthesized/estimated area; sparse metadata overhead; and model accuracy after pruning.
- Method: use a cycle-level framework with DRAM/layout support such as SCALE-Sim v3 concepts; validate key kernels in RTL, HLS, or FPGA where possible. Never use ideal infinite-bandwidth results as the main conclusion.

## 6. Reference list

- [1] Google Cloud, "An in-depth look at Google's first Tensor Processing Unit (TPU)," 2018.  
<https://cloud.google.com/blog/products/ai-machine-learning/an-in-depth-look-at-googles-first-tensor-processing-unit-tpu>
- [2] H. B. et al., "Systolic Array Data Flows for Efficient Matrix Multiplication," arXiv:2410.22595, 2024. <https://arxiv.org/html/2410.22595v1>
- [3] Y.-H. Chen et al., "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," IEEE JSSC, 2016.  
<https://ieeexplore.ieee.org/document/7738524>
- [4] Y. Kwon et al., "MAERI: Enabling Flexible Dataflow Mapping over DNN Accelerators via Reconfigurable Interconnections," ASPLOS, 2018.  
<https://mlsys.org/Conferences/doc/2018/34.pdf>
- [5] M. K. et al., "Sparse-TPU: Adapting Systolic Arrays for Sparse Matrices," ICS, 2020.  
<https://nyuscholars.nyu.edu/en/publications/sparse-tpu-adapting-systolic-arrays-for-sparse-matrices>
- [6] A. S. et al., "SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training," HPCA, 2020.  
<https://www.computer.org/csdl/proceedings-article/hpca/2020/614900a058/1j9wsY4zRmM>
- [7] A. C. Yüzügüler et al., "Scale-out Systolic Arrays," ACM TACO, 2023; preprint arXiv:2203.11540. <https://arxiv.org/pdf/2203.11540.pdf>
- [8] "DiP: A Scalable, Energy-Efficient Systolic Array for Matrix Multiplication," arXiv:2412.09709, revised 2025. <https://arxiv.org/html/2412.09709v2>
- [9] "FlexiSAGA: A Flexible Systolic Array GEMM Accelerator for Sparse and Dense DNNs," arXiv:2506.01566, 2025.  
<https://arxiv.org/html/2506.01566v1>
- [10] "A modular cycle-accurate systolic accelerator simulator for end-to-end DNN performance analysis," arXiv:2504.15377, 2025.  
<https://arxiv.org/html/2504.15377v1>
- [11] "Addressing memory bandwidth scalability in vector processors for data-parallel AI/ML," arXiv:2505.12856, 2025.  
<https://arxiv.org/html/2505.12856v1>
- [12] EPFL Embedded Systems Laboratory, "Systolic Array Accelerators for Transformers (TiC-SAT)," 2024.  
<https://www.epfl.ch/labs/esl/research/full-system-simulation-and-design/systolic-array-accelerators-for-transformers/>

**Source-quality note.** References [1], [3]–[7] include industry, conference, or journal sources. References [2], [8]–[11] are preprints and should be checked for final peer-reviewed versions before being relied upon as settled evidence.